

TRABAJO FINAL INTEGRADOR

PROGRAMACIÓN CON OBJETOS 2

Integrantes:

- **Tchurukian, Martin:** mat34218@gmail.com
- **Ibarra, Sofia:** sofiaibarra415@gmail.com
- **Fernandez, Martin:** martxnfernandxz101@gmail.com

Repositorio Github:

<https://github.com/sofiaibarra415/tp-integrador-ibarra-tchurukian-fernandez/tree/master/src/eccomerce>

2.1 Catálogo de productos

Para este punto se decidió utilizar la clase **Catálogo**. Esta contiene una lista de elementos de la clase abstracta **Ítem**, que tiene dos subclases concretas llamadas **Producto** y **Paquete**. Además los ítems tienen una lista de elementos de la clase **Atributo** para los atributos dinámicos. El patrón de diseño utilizado es COMPOSITE con los siguientes roles:

- Componente : Ítem.
- Compuesto: Paquete.
- Hoja: Producto.

Esto se decidió para cumplir con el requisito de que paquetes y productos compartan contrato.

Finalmente se tomaron las siguientes decisiones de diseño:

- Peso sea un atributo necesario y no dinámico.
- Un paquete tiene peso sólo si todos sus componentes lo tienen y es la suma de estos (no un valor seteable).
- El stock de un paquete es el del producto que lo compone con menor stock.
- Utilizar una clase atributo por tratarse de un trabajo práctico de POO (con un diccionario se solucionaba).

2.2 Ciclo de vida del pedido

Para este punto del trabajo decidimos usar el patrón STATE. Un **Pedido** conoce su Estado y al mismo tiempo un Estado conoce su Pedido. La clase abstracta **EstadoPedido** tiene subclases que son los **Estados Concretos**. Cuando el pedido recibe una actualización de su estado, el mismo le manda un mensaje al estado para que actúe. Como no todos los estados pueden hacer todo, por defecto en la clase abstracta de estado se arroja una **excepción** en todos los cambios de estado, luego son los estados concretos quienes redefinen lo que les corresponde. Cuando un pedido se cancela desde ciertos estados, se genera una **NotaDeCredito** registrada en el pedido con el monto correspondiente al reembolso

nota: quizás un estado no necesita conocer al pedido (tenerlo como variable de instancia) y simplemente se puede pasar como parámetro. Fue una decisión de comodidad pero podría ser un refactor.

Un detalle importante: Pedido.setEstado() es el único punto de transición

Aquí los roles son:

- EstadosConcretos : Borrador, Confirmado, EnPreparacion, etc.
- Estado: clase abstracta EstadoPedido
- Contexto: el pedido, quien manda estado.actuar()

2.3 Métodos de envío

Para este punto del trabajo decidimos usar el patrón STRATEGY. Un pedido conoce su **MetodoEnvio**, y el costo del mismo se calcula distinto según el método de envío elegido. Puede ser **Express**, **Estándar** o **Retiro en Sucursal**. Lo mismo pasa con estimar los días de envío, depende del método de envío. Pedido no sabe cómo se calcula nada de esto. A pedido se le setea un método de envío, y a partir de ahí se sigue una estrategia particular. Para EnvioEstandar y EnvioExpress, el enunciado nos daba librerías externas. Las definimos como interfaces propias (**CorreoArgentinaAPI**, **EnvioExpressAPI**) que se inyectan por constructor, lo que nos permite testear sin depender de implementaciones externas

Retiro en Sucursal necesita conocer una **Sucursal** para preguntarle si hay stock en el local, EnvioEstandar tiene una **Dirección** para pasarsela a la interfaz CorreoArgentina y determinar el precio de envío.

nota: en ningún lado pedía que para confirmar exista un método de envío pero decidimos agregar esta validación porque es lo que pasa en un ecommerce en la vida real.

Aquí los roles son:

- Contexto: Pedido
- Estrategia: MetodoEnvio
- Estrategia Concreta: EnvioExpress, EnvioEstandar, RetiroEnSucursal

2.4 Métodos de pago

Para este punto del trabajo se decidió usar el patrón TEMPLATE METHOD. La clase abstracta **MetodoDePago** define cuatro funciones que reflejan los cuatro pasos del procesamiento de un pago, las primeras tres siendo abstractas: validación de los datos, reserva de fondos, ejecución de la transacción y notificación del resultado. Estas son heredadas por las subclases **TarjetaDeCredito**, **TransferenciaBancaria** y **BilleteraVirtual**, las cuales poseen su propia implementación para cada una de dichas funciones. A la hora de procesar un pago, MetodoDePago posee una función que ejecuta los cuatro pasos en orden.

Se tomó la (controversial) decisión de utilizar *data classes* para el manejo de los diferentes tipos de datos que reciben los métodos de pago. Si bien se sabe que este tipo de clases tienen *bad smell*, se les asignó el rol de verificar que el formato en el que se insertan datos sea correcto. Por ejemplo, el CVV debe ser un número entre 3 y 4 caracteres, de lo contrario, es un dato inválido. **DatosPago** es una interfaz implementada por los tres tipos de datos de pago correspondientes a cada método, **DatosTarjetaCredito**, **DatosTransferenciaBancaria** y **DatosBilleteraVirtual**, quienes poseen la lógica de verificación de datos. Básicamente, es como una pre-validación de datos antes de comunicarse con la API, quien valida con su propia lógica externa.

Con respecto a las APIs, se definieron tres interfaces: **APITarjeta**, **APIBanco** y **APIBilletera**, que son usadas en la mayoría de los pasos. Ellas tienen las firmas que utilizan los métodos de pago para comunicarse con el “mundo exterior”, pero en este caso el mundo exterior es el test de unidad con los mocks, que devuelven datos ya programados.

El último paso del procesamiento de un pago requiere, en la mayoría de casos, registrar un comprobante, el cual es guardado en una lista en **RegistroPagos**. Para el caso

de TarjetaDeCredito, el comprobante es un **CuponPago**. Para TransferenciaBancaria, **ComprobanteCBU**. BilleteraVirtual no requiere registrar ningún comprobante, solo enviar una notificación push. Sin embargo, el enunciado indica que este paso tiene una implementación por defecto que registra el código de transacción, la cual no es utilizada por ninguna clase, pero debe ser implementada de todas formas. Para solucionarlo, se decidió crear otra lista en RegistroPagos que los guarde y, para respetar el hecho que ya existe dicha estructura, todos los métodos de pago también registran el código de transacción.

Los roles del template method que emplean las clases son:

- Clase abstracta: MetodoDePago.
- Clases concretas: TarjetaDeCredito, TransferenciaBancaria y BilleteraVirtual.

2.5 Notificaciones del Pedido

Para este punto del trabajo decidimos usar el patrón OBSERVER. Un pedido conoce una lista de observadores los cuales reaccionan siempre que el pedido cambie de estado. Pedido conoce una lista de **ObserverPedido** pero no sabe quiénes son ni qué hacen. Cuando pedido responde el mensaje setEstado() a su vez notifica a todos sus observadores para que actualicen con el estado anterior y el actual. Los observadores concretos deciden si actúan dependiendo de si les corresponde.

nota: validamos poner mensajes como "esAlertable()" o esFacturable() para no poner directamente el estado concreto al que aplica. Así se mantiene encapsulada en los estados la lógica para decidir quién es alertable o no.

Además, usamos la interfaz **MailSender** para separar la lógica de mandar el mail.

Aquí los roles son:

- Sujeto: Pedido
- Observador: ObserverPedido
- Sujeto Concreto: en este caso no hay
- Observador Concreto: **NotificadorDeEmail**, **GeneradorDeFactura**, **Fidelizacion**

nota: podríamos haber usado una clase abstracta Sujeto y una clase Pedido (Sujeto Concreto) que extienda de Sujeto para luego poner otros sujetos concretos a futuro pero para este trabajo el pedido solo necesitaba observers, era lógica que no íbamos a necesitar.

2.6 Búsqueda en el catálogo

Para este punto se decidió utilizar el patrón de diseño COMPOSITE, ya que hay criterios de búsqueda tanto simples como compuestos que deben funcionar de forma polimórfica. Los roles son los siguientes:

- Componente :
 - **CriterioDeBusqueda**
- Compuesto:

- **CriterioDeBusquedaCompuestoAND**
 - **CriterioDeBusquedaCompuestoOR**
 - **CriterioDeBusquedaCompuestoNOT**
- Hoja:
 - **CriterioDeBusquedaSimpleNombre**
 - **CriterioDeBusquedaSimpleDisponibilidad**
 - **CriterioDeBusquedaSimpleCategoria**
 - **CriterioDeBusquedaSimplePrecioMaximo**

Finalmente se tomaron las siguientes decisiones de diseño:

- Los compuestos para AND y OR admiten una cantidad dinámica de componentes.
- En caso que la lista de componentes de los compuestos para AND y OR sea vacía “evaluar(Item)” retorna el booleano neutro para la operación. La interfaz gráfica no debería crear un criterio compuesto vacío, pero como no la programamos, se decidió esta salvaguarda.

2.8 Reportes

Este punto pedía usar el patrón de diseño Visitor. Este se implementó con los siguientes roles:

- Visitante :
 - **ItemVisitor**
- Visitantes concretos:
 - **ReporteTextoPlanoVisitor**
 - **ReporteCSVVisitor**
 - **ReporteHTMLVisitor**
- Elemento:
 - **Item**
- Elementos concretos:
 - **Producto**
 - **Paquete**
- Estructura de Objetos:
 - **Catalogo**

Además fue necesario crear una clase **Venta** que registre las ventas de los productos y paquetes. Finalmente se tomaron las siguientes decisiones de diseño:

- ItemVisitor es una superclase abstracta (en vez de una interfaz) para no repetir la lógica compartida de los visitors concretos.
- Que la lógica recursiva del cálculo de precios quede encapsulada en **Paquete**.
- Mantener el double dispatch del patrón Visitor a pesar de no ser necesario gracias a que la lógica recursiva quedó dentro de **Paquete**.